Chapter 16 — Communication & Career

16.1 Translating Business Metrics to ML Metrics

Conceptual Deep Dive

Stakeholders care about revenue, cost, risk and time saved; models produce accuracy, faithfulness and latency. Build a **metric tree**: business outcome (e.g. support cost per ticket) → product metric (deflection rate, CSAT, handle time) → system metric (answer faithfulness, retrieval recall@5, escalation precision) → model/infra metric (latency, token cost). Choose the metric that matches error costs: for a refund bot, a false approval costs far more than a false denial, so weight precision accordingly. Convert to money: `value = volume × (success_rate × benefit − failure_rate × cost_of_failure) − run_cost`. Always state a baseline (current human or rule-based process) and a decision threshold ("ship if deflection ≥ 30% with CSAT drop ≤ 1 point").

Cross-Connection Scenarios

Evaluation design (10), A/B testing (10.5), cost modelling (2.4, 13.5).

Production Anti-Patterns

- Reporting "92% accuracy" without saying on what, versus what baseline, at what cost of errors.
- Optimising a proxy metric the business doesn't care about.
- Promising deterministic behaviour from a probabilistic system.

Minimal Working Code

```
def roi(volume, success_rate, benefit, failure_rate, failure_cost, run
_cost_per_req, fixed=0.0):
    gain = volume * (success_rate * benefit - failure_rate * failure_cost)
    cost = volume * run_cost_per_req + fixed
    return {"net_monthly": gain - cost, "roi": (gain - cost) / cost if cost else floa
t("inf")}

print(roi(volume=100_000, success_rate=0.7, benefit=4.0, failure_rate=0.05,
          failure_cost=20.0, run_cost_per_req=0.03, fixed=8_000))
```

16.2 Communicating with Stakeholders

Conceptual Deep Dive

Explain uncertainty honestly: "right about 9 in 10 times on our test set; here is what the other 1 looks like and how we catch it." Use concrete examples (good, typical, worst case) over averages. Set expectations on timelines: prototypes are quick, the last 20% (evals, safety, edge cases) takes most of the time. Write short decision documents (problem, options, recommendation, risks, cost, metrics). Run demos with failure cases included. Create a shared definition of "done" and an escalation path. For executives: outcome, risk, cost, timeline in one page. For legal/security: data flows, retention, vendors, controls. For users: what it can and cannot do, and how to give feedback.

Cross-Connection Scenarios

Risk controls (14), phased rollouts (15.5), roadmap planning (17, 18).

Production Anti-Patterns

- Cherry-picked demos that set impossible expectations.
- Jargon without translation.
- Hiding limitations until launch.

Minimal Working Code

```
TEMPLATE = """# Decision: {title}
**Problem:** {problem}
**Options considered:** {options}
**Recommendation:** {rec}
**Expected impact:** {impact}
**Risks and mitigations:** {risks}
**Cost (monthly):** {cost}
**Success metrics / ship criteria:** {criteria}
"""

def render(**kw) -> str:
    return TEMPLATE.format(**kw)

print(render(title="Adopt RAG for support", problem="High ticket volume", options=&qu
ot;FAQ bot, RAG, fine-tune",
             rec="RAG with human escalation", impact="-25% tickets", risks="hall
ucination -> citations + abstain",
             cost="$6k", criteria="faithfulness>=0.9, CSAT drop<=1pt"))
```

16.3 System Design Interviews for AI Engineers

Conceptual Deep Dive

Framework: (1) clarify requirements and scale (users, QPS, latency, accuracy, privacy), (2) define success metrics, (3) sketch data flow, (4) deep-dive key components (retrieval, model choice, caching, guardrails), (5) address evaluation and monitoring, (6) cost and scaling, (7) failure modes and trade-offs. Common prompts: design a RAG chatbot over enterprise docs; design a code assistant; design an LLM gateway; design semantic search; design a content-moderation pipeline; design a multi-agent research system; design an evaluation platform; ML-ops for fine-tuning and serving. Show trade-off fluency: HNSW vs IVF-PQ, API vs self-host, workflow vs agent, chunk size vs context, latency vs quality, cache vs freshness. Do back-of-envelope math (tokens/day, cost, GPU count, index memory).

Cross-Connection Scenarios

Every chapter appears; use Chapter 19 for quick numbers and Chapter 17 for reference architectures.

Production Anti-Patterns

- Jumping to tools before requirements.
- Ignoring evaluation and safety.
- No capacity numbers.

Minimal Working Code

```
def capacity_estimate(dau, req_per_user, tin, tout, tok_per_s_per_gpu,
 peak_factor=3, seconds=86400):
    daily_reqs = dau * req_per_user
    peak_rps = daily_reqs / seconds * peak_factor
    out_tokens_per_s = peak_rps * tout
    return {"peak_rps": round(peak_rps, 2),
            "gpus": max(1, round(out_tokens_per_s / tok_per_s_per_gpu + 0.5)),
            "monthly_tokens_in": daily_reqs * tin * 30, "monthly_tokens_out": daily_r
eqs * tout * 30}
```

```
print(capacity_estimate(dau=50_000, req_per_user=5, tin=2000, tout=400, tok_per_s_per_gpu=2500))
```

16.4 Career Progression as an AI Engineer

Conceptual Deep Dive

Levels: **junior** (implements scoped features, writes tests, learns the stack) → **mid** (owns a component end to end, designs evals) → **senior** (owns systems, sets technical direction, mentors, makes cost/risk trade-offs) → **staff/principal** (cross-team architecture, platform strategy, influences org-level decisions). Differentiators: shipping to production, measurable impact, evaluation rigour, security awareness, and clear communication. Specialisation paths: applied/product AI engineer, ML platform/infra (serving, GPUs), evaluation & safety, fine-tuning/research engineer, AI product management, developer relations. Build a portfolio: 3 shipped projects with README, architecture diagram, eval results, cost analysis, and a failure-analysis write-up. Keep learning via papers (arXiv), release notes, open-source contributions, and incident postmortems. The field changes quickly; invest in fundamentals (Chapters 3–5, 10) that outlast any framework.

Cross-Connection Scenarios

Learning plan (18), projects (17).

Production Anti-Patterns

- Collecting certificates instead of shipping systems.
- Chasing every new framework.
- Portfolio of notebooks with no deployment or evaluation.

Minimal Working Code

```
LADDER = {
    "junior": ["ships scoped features", "writes tests", "uses evals&qu
ot;],
    "mid": ["owns a component", "designs eval sets", "debugs productio
n issues"],
    "senior": ["owns system design", "balances cost/quality/risk", "me
ntors"],
    "staff": ["sets cross-team architecture", "defines platform strategy",
"drives org-level metrics"],
}

def gap_analysis(level: str, have: set[str]) -> list[str]:
    return [s for s in LADDER[level] if s not in have]

print(gap_analysis("mid", {"owns a component"}))
```

16.5 Scoping, Estimating and Managing AI Projects

Conceptual Deep Dive

AI projects fail from unclear success criteria more often than from weak models. Scope with a **one-page spec**: user, task, input/output examples, success metric and threshold, failure cost, data availability, latency/cost budget, and rollout plan. Run a **feasibility spike** (1–3 days): hand-test 30 real examples with the strongest model to learn the achievable ceiling before committing. Estimate in phases with explicit uncertainty: prototype (days), eval-driven iteration (weeks), hardening and safety (weeks), rollout and monitoring (ongoing). Manage risk with kill criteria ("if recall@5 is below 0.7 after three iterations, we pivot"). Edge cases: stakeholders anchoring on a flashy demo, scope creep into "just add one more tool", and data access delays (legal, security) dominating the schedule.

Cross-Connection Scenarios

Metrics translation (16.1), evaluation (10), roadmap projects (17), CI gates (15).

Production Anti-Patterns

- Committing to a deadline before a feasibility spike.
- No kill criteria, so failing projects continue for months.
- Treating evaluation as the final week's task.

Minimal Working Code

```
def feasibility(examples: list[dict], solve, judge, ceiling_target: fl
oat = 0.8) -> dict:
    passed = sum(judge(ex, solve(ex["input"])) for ex in examples)
    rate = passed / len(examples)
    return {"pass_rate": rate, "decision": "proceed" if rate >= ceiling_
target else
            "pivot or collect more context/data"}

print(feasibility([{"input": 2, "expected": 4}], lambda x: x * 2, lambda ex, out: out
 == ex["expected"]))
```

```
from dataclasses import dataclass

@dataclass
class Phase:
    name: str
    best_weeks: float
    likely_weeks: float
    worst_weeks: float

    @property
    def expected(self) -> float:
        return (self.best_weeks + 4 * self.likely_weeks + self.worst_weeks) / 6   # PERT

plan = [Phase("prototype", 0.5, 1, 2), Phase("eval-driven iteration", 2, 4, 8), Phase
("hardening", 2, 3, 6)]
print(round(sum(p.expected for p in plan), 1), "weeks expected")
```

16.6 Writing, Documentation and Incident Communication

Conceptual Deep Dive

Good AI engineers write clearly. Artifacts that pay off: **design docs** (context, goals/non-goals, options, decision, risks), **model/system cards** (intended use, data, eval results by slice, limitations, safety measures), **runbooks** (symptom, diagnosis, mitigation, escalation), **eval reports** (setup, results with intervals, failure analysis, recommendation), and **blameless postmortems** (timeline, impact, root cause, contributing factors, action items with owners). For incidents, communicate early with what is known, what is unknown, current mitigation and next update time. For users, document limitations plainly and show how to give feedback. Edge cases: communicating model-version changes that alter behaviour, disclosing AI-generated content where required, and handling a harmful output screenshot circulating publicly.

Cross-Connection Scenarios

Governance (14.7), regression suites from postmortems (15.2), career visibility (16.4).

Production Anti-Patterns

- Postmortems that name individuals instead of fixing systems.
- Model cards with only aggregate accuracy.
- Silence during an incident until the root cause is known.

Minimal Working Code

```
POSTMORTEM = """# Postmortem: {title}
**Impact:** {impact}
**Timeline:** {timeline}
**Root cause:** {root}
**Contributing factors:** {factors}
**What went well:** {good}
**Action items:** {actions}   (each with owner + due date)
"""

def postmortem(**kw) -> str:
    missing = {"title", "impact", "timeline", "root", "facto
rs", "good", "actions"} - kw.keys()
    if missing:
        raise ValueError(f"missing sections: {sorted(missing)}")
    return POSTMORTEM.format(**kw)
```

```
def model_card_table(slices: dict[str, dict]) -> str:
    rows = ["| Slice | n | Pass rate | 95% CI |", "|---|---|---|---|"]
    for name, s in slices.items():
        rows.append(f"| {name} | {s['n']} | {s['p']:.2f} | {s['lo']:.2f}–{s['hi']:.2f} |")
    return "\n".join(rows)

print(model_card_table({"English": {"n": 400, "p": .91, "lo": .88
, "hi": .93},
                        "Spanish": {"n": 60, "p": .83, "lo": .72,
 "hi": .91}}))
```